Design Rationale for REQ4

The UML and Sequence Diagrams for REQ4 can be found in the same directory.

Functionality
The Purchasable interface defines the contract for any class implementing it to require the methods getCost(), bugChance(), bug() and purchaseEffect(Actor actor).

The PurchaseAction class that extends Action allows items such as EnergyDrink, DragonSlayerSword and ToiletPaperRoll to be purchased by an actor.

The EnergyDrink class that extends Item and implements Consumable and Purchasable can be purchased and consumed by the player to heal their HP by 1. There is a 20% chance of bugging which will cause the $10 EnergyDrink to cost double the original price ($20).

The DragonSlayerSword class that extends WeaponScrap and implements Purchasable can be purchased by the player and used as a weapon to deal 50 damage with a hit rate of 75%. There is a 50% chance of bugging which will cause the terminal to accept the Player's money ($100) without printing the DragonSlayerSword.

The ToiletPaperRoll class that extends Item and implements Purchasable can be purchased by the player but does nothing. There is a 75% chance of bugging which will cause the the $5 ToiletPaperRoll to cost only $1.

The Terminal class that extends Ground allows an actor to access it and purchase items.

Object-Oriented Design Principles
Every class only contains their respective capabilities. This means that they are responsible for only one part of the program's functionality. (Single Responsibility Principle)

Pros:
- Bugs will be easy to find as each class deals with a specific function of the system.

Cons:
- There may be an abundance of classes which could make maintaing the system difficult

The Purchasable interface allows future purchasable items to simply implement the interface without altering existing code. This keeps it open for extension but closed for modification. (Open-closed Principle)

Pros:
- The system is more extensible

Cons:
- If the interface requires modification, every file implementing the interface will have to be refactored

Since the child classes extend their respective parent classes (e.g. EnergyDrink extends Item), which define the common behaviour of all their subclasses, the subclasses can be used in place of their respective parent classes without disrupting the behaviour of the program. (Liskov Substitution Principle)

Pros:
- The system is more flexible and achieves polymorphism

Cons:
- Subclasses overriding parent classes unnecessarily could cause the performance to suffer

Every purchasable item that implements the Purchasable interface care about every method in the interface. This means that no item is implementing methods that it does not need. (Interface Segregation Principle)

Pros:
- The purchasable items only contain relevant methods which will avoid unnecessary implementations

Cons:
- There could be an excessive amount of interfaces which could make it difficult to maintain the system

The concrete child classes do not depend on another concrete class but on the abstract parent classes instead (e.g. ToiletPaperRoll class extends Item abstract class). (Dependency Inversion Principle)

Pros:
- Loose coupling allows for easier modification and texting which makes the system more flexible

Cons:
- Too many abstract classes could overcomplicate the system which may make it difficult to maintain the system

Justification
Despite the potential increased complexity, the SOLID principles allows for easier modification and extensibility.